Department of Computer Science & Engineering
The University of Texas at Arlington

Architectural Design Specification
CSE 4316: Senior Design I
Fall 2025

ProtoBot
Starcraft Brood War Game Playing Agent

Preston Loera
Matthew Nguyen
Marshall Helton
David Daffern
Aadhitya Kumar
Tung Nguyen

Revision History

Revision	Date	Author(s)	Description
0.1	10.27.2025	AK, TN	document creation
1.0	10.31.2025	TN, AK, PL, MN, MH, DD	official release
2.0	4.8.2026	AK, TN	document update
3.0	4.10.2026	PL, MN	document update

Contents

1 Introduction **6**

2 System Overview **7**
2.1 Commander . . . 7
2.2 Economy Manager . . . 7
2.3 Build Manager . . . 7
2.4 Strategy Manager . . . 7
2.5 Scout Manager . . . 8
2.6 Information Manager . . . 8
2.7 Combat Manager . . . 8

3 Subsystem Definitions & Data Flow **9**

4 ProtoBot Commander **10**
4.1 Economy . . . 10
4.2 Build . . . 12
4.3 Scouting . . . 14
4.4 Information . . . 16
4.5 Strategy . . . 17
4.6 Combat . . . 18
4.7 A* Path Generation . . . 19
4.8 Timer Manager . . . 20

5 Economy Manager **21**
5.1 Nexus Economy . . . 21
5.2 Geysers . . . 23
5.3 Minerals . . . 23
5.4 Workers . . . 24

6 Build Manager **25**
6.1 Build Order . . . 25
6.2 BWEB Module . . . 26
6.3 Building Placer . . . 27

7 Scout Manager **28**
7.1 Scout Module . . . 28
7.2 BWEM Module . . . 29

8 Information Manager **30**
8.1 Unit/Building Tracker . . . 30
8.2 Game State Tracker . . . 30

9 Strategy Manager **32**
9.1 Strategy State . . . 32
9.2 Production Goal . . . 33
9.3 Spender Manager . . . 34

10 Combat Manager **35**
10.1 Squads 35
10.2 Squad State 37

List of Figures

1 Architectural Layers for the Starcraft Game Playing Agent . . . 7
2 Data flow for a BWAPI OnFrame Event for ProtoBot . . . 9
3 Economy subsystem diagram . . . 10
4 Build subsystem diagram . . . 12
5 Scouting subsystem diagram . . . 14
6 Information subsystem diagram . . . 16
7 Strategy subsystem diagram . . . 17
8 Combat subsystem diagram . . . 18
9 A* Path Generation subsystem diagram . . . 19
10 A* Path Generation subsystem diagram . . . 20
11 Nexus Economy subsystem diagram . . . 21
12 Geysers subsystem diagram . . . 23
13 Mineral subsystem diagram . . . 23
14 Worker subsystem diagram . . . 24
15 Build Order subsystem diagram . . . 25
16 BWEB Module subsystem diagram . . . 26
17 Building Placer subsystem diagram . . . 27
18 Scout Module subsystem diagram . . . 28
19 BWEM Module subsystem diagram . . . 29
20 Unit/Building Tracker subsystem diagram . . . 30
21 Game State Tracker subsystem diagram . . . 31
22 Strategy State subsystem diagram . . . 32
23 Production Goal subsystem diagram . . . 33
24 Spender Manager subsystem diagram . . . 34
25 Combat Module subsystem diagram . . . 35
26 Squad Module subsystem diagram . . . 37

List of Tables

2 Economy subsystem interfaces . . . 11
3 Build subsystem interfaces . . . 13
4 Scouting subsystem interfaces . . . 15
5 Information subsystem interfaces . . . 16
6 Strategy subsystem interfaces . . . 17
7 Combat subsystem interfaces . . . 18
8 Nexus Economy subsystem interfaces . . . 22
9 Workers subsystem interfaces . . . 24
10 Building Placer subsystem interfaces . . . 27
11 Scouts subsystem interfaces . . . 28
12 Unit/Building Tracker subsystem interfaces . . . 30
13 Game State Tracker subsystem interfaces . . . 31
14 Spender Manager subsystem interfaces . . . 34
15 Squad Module subsystem interfaces . . . 36

1 Introduction

This document presents the Architectural Design Specification (ADS) for ProtoBot, an autonomous *StarCraft: Brood War* game-playing agent designed to demonstrate intelligent and adaptive decision-making in the *StarCraft: Brood War* real-time environment. ProtoBot uses rule-based decision-making to manage tasks such as resource management, build execution, scouting, and combat. These decisions allow ProtoBot to adapt its behavior based on the state of the game and opponent's actions. Our bot makes use of multiple managers, each handling an aspect of the game. For each manager to receive events about the current state of a StarCraft game, a commander class known as the "ProtoBot Commander" sends BWAPI events and ProtoBot information for managers to make decisions based on the information given. Through this structure, ProtoBot can adapt its strategy in real time to win against other bots.

2 System Overview

The software structure of ProtoBot is divided into six distinct managers: Economy Manager, Build Manager, Scout Manager, Information Manager, Strategy Manager, and Combat Manager. Each manager is comprised of multiple subsystems, each with unique functionalities that help ProtoBot make decisions about a specific task. The high-level block diagram below describes ProtoBot's modular structure.

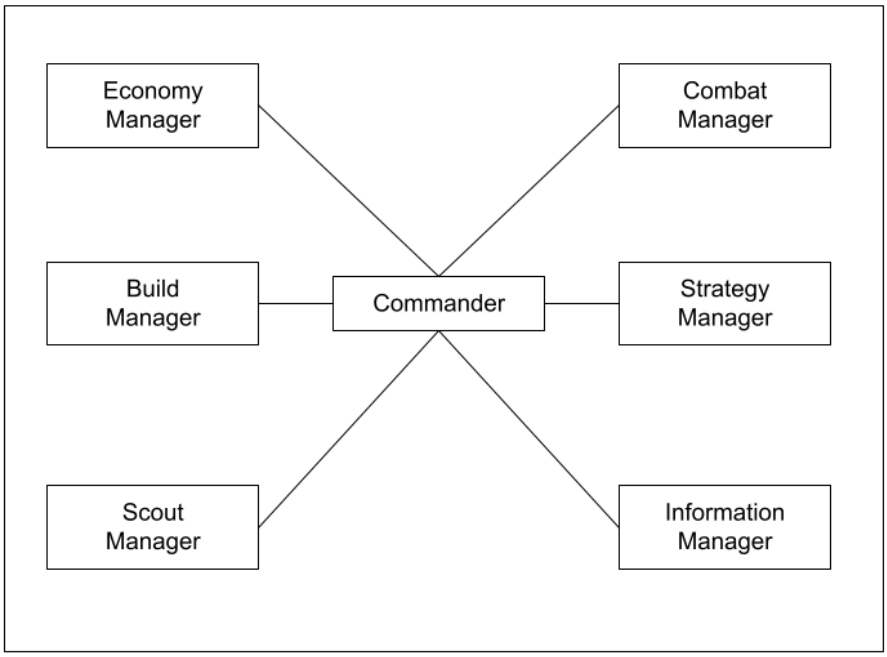

Figure 1: Architectural Layers for the Starcraft Game Playing Agent

2.1 Commander

The Commander is the central module of the bot, receiving and passing BWAPI events to all managers and allowing communication between subsystems if necessary. This module ensures that each manager operates on up-to-date game information and that their states can be updated by information from another module if needed.

2.2 Economy Manager

The Economy Manager handles the harvesting of resources and management of workers.

2.3 Build Manager

The Build Manager specializes in setting up build orders and determining placements of buildings.

2.4 Strategy Manager

The Strategy Manager determines production goals and high-level decision-making based on the current game state.

2.5 Scout Manager

The Scout Manager executes scouting routes/policies, updating the information found from scouting to other modules.

2.6 Information Manager

The Information Manager keeps track of the current game state and recognizes enemy's build order/strategy.

2.7 Combat Manager

The Combat Manager controls squads of combat units in order to attack and defend positions on the map.

3 Subsystem Definitions & Data Flow

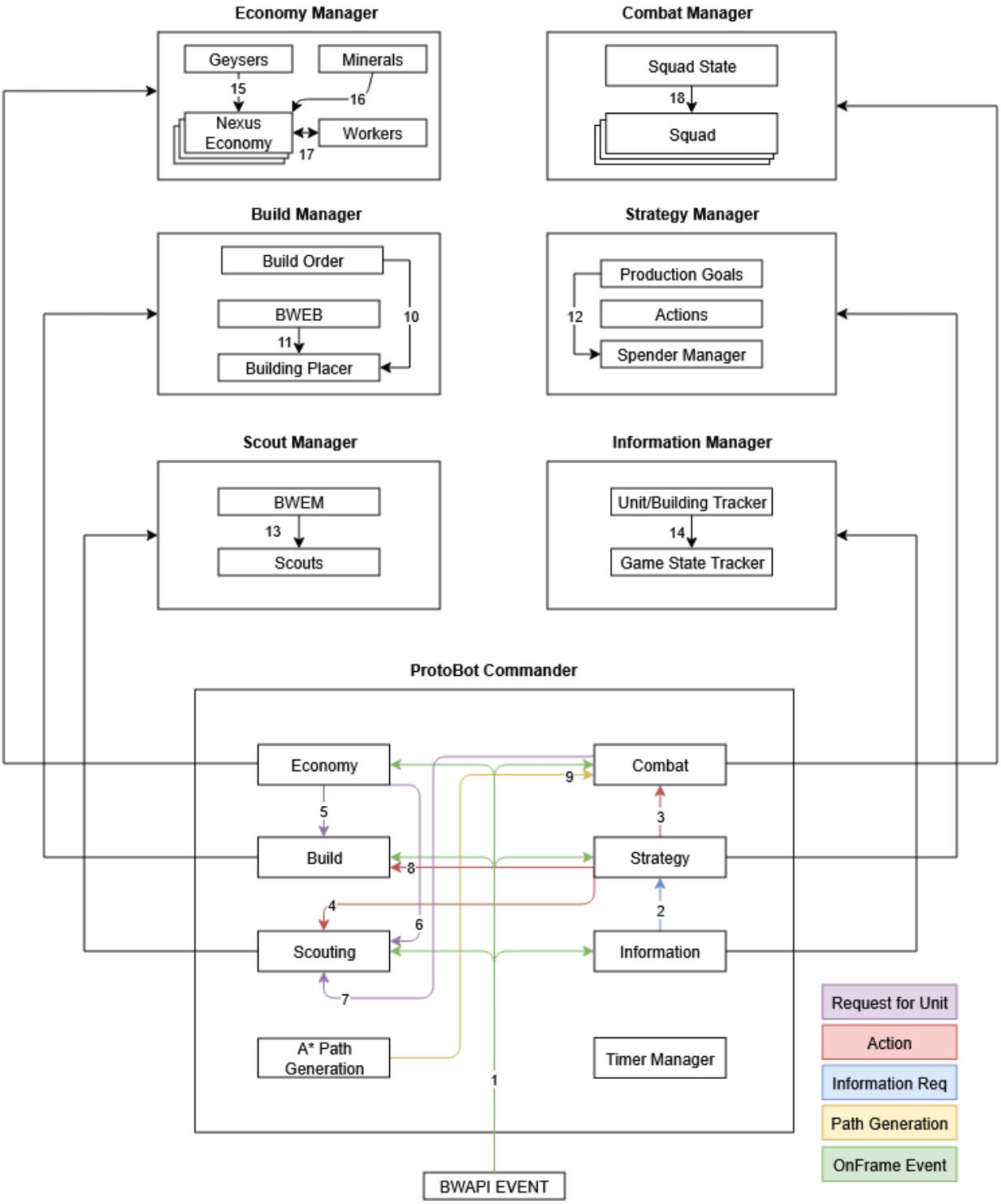

Figure 2: Data flow for a BWAPI OnFrame Event for ProtoBot

4 ProtoBot Commander

The main purpose of the ProtoBot Commander is to forward the events created by BWAPI to each manager. The commander has no real intelligence and is solely responsible for communicating actions sent from the Strategy manager to other managers or updating data structures for further processing. Any actions sent by the Strategy Manager will then be executed by the manager responsible for executing said action.

4.1 Economy

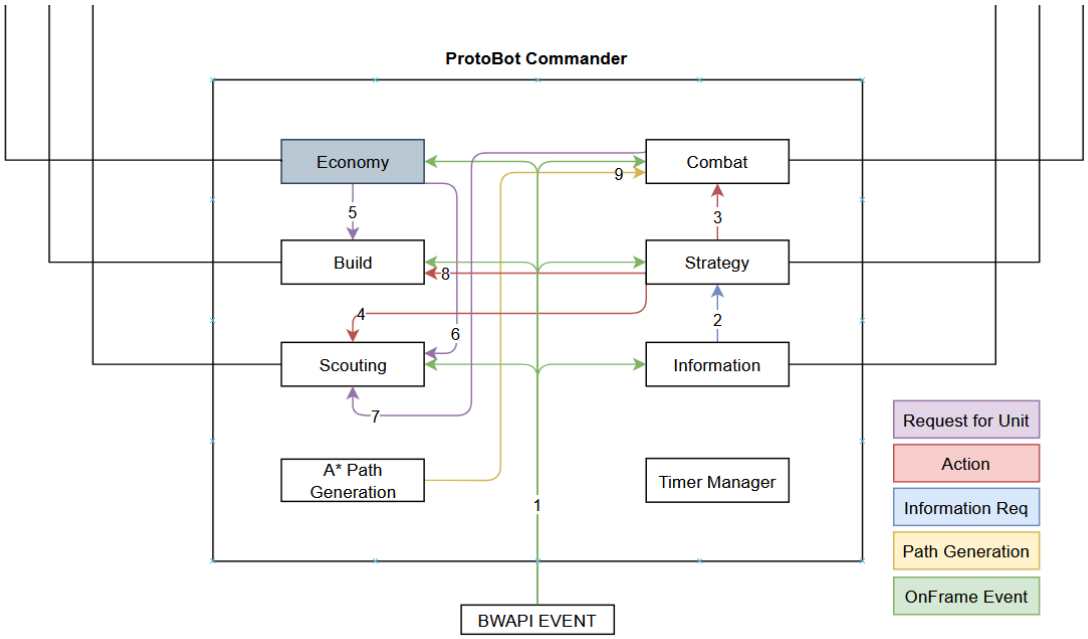

Figure 3: Economy subsystem diagram

4.1.1 Assumptions

- Any completed worker will be sent to the Economy Manager to be assigned to a base
- The Economy Manager will optimize resource gathering

4.1.2 Responsibilities

- Pass completed workers to the Economy Manager
- Pass completed nexuses to Economy Manager
- Pass completed assimilator's to Economy Manager
- Send BWAPI events to Economy Manager

4.1.3 Subsystem Interfaces

Table 2: Economy subsystem interfaces

ID	Description	Inputs	Outputs
#1	StarCraft BWAPI Event	BWAPI Event	Action(s) to perform optimal resource farming and reassignment of workers as necessary

4.2 Build

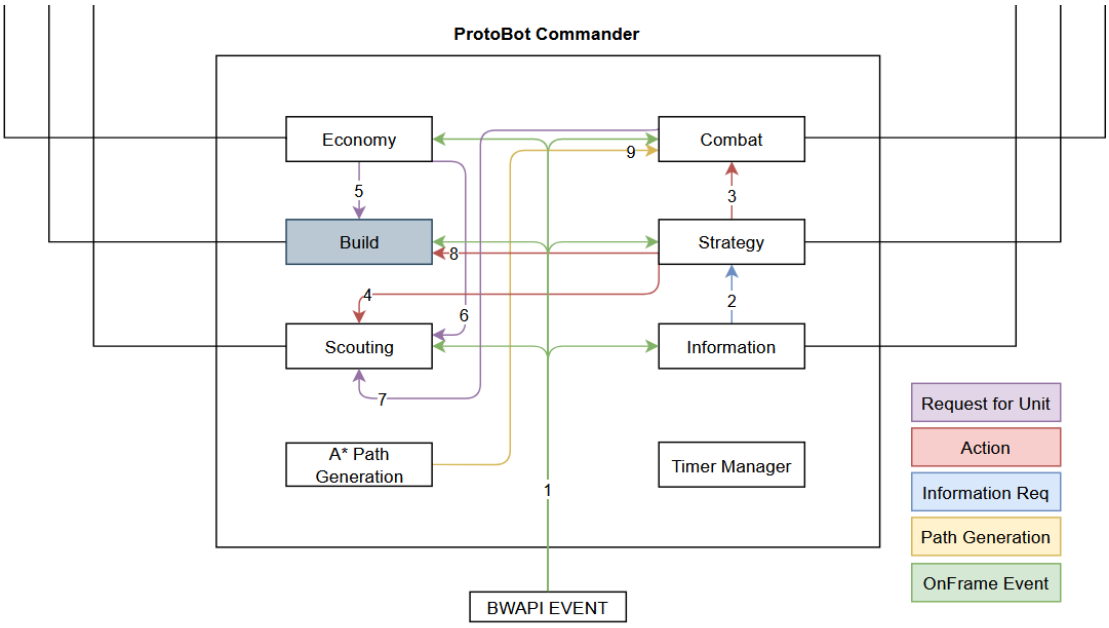

Figure 4: Build subsystem diagram

4.2.1 Assumptions

- The required information for the creation of units, buildings, and upgrades will be specified for the Build Manager to service a resource request
- The Strategy Manager will provide valid instructions for resource requests

4.2.2 Responsibilities

- Provide information about the current resource requests ProtoBot needs to create
- Mark a resource request as completed or in progress once further information is added
- Send BWAPI events to Build Manager

4.2.3 Subsystem Interfaces

Table 3: Build subsystem interfaces

ID	Description	Inputs	Outputs
#1	StarCraft BWAPI Event	BWAPI Event	Actions to add items to perform a build order and process resource requests
#5	Request for worker unit to construct a unit	Worker (Protoss Probe)	Assignment of a builder to construct a building a specified tile position
#6	Requests to create specific units	Resource request from Strategy Manager/Commander	Confirmation of units being created and servicing of builders to construct buildings

4.3 Scouting

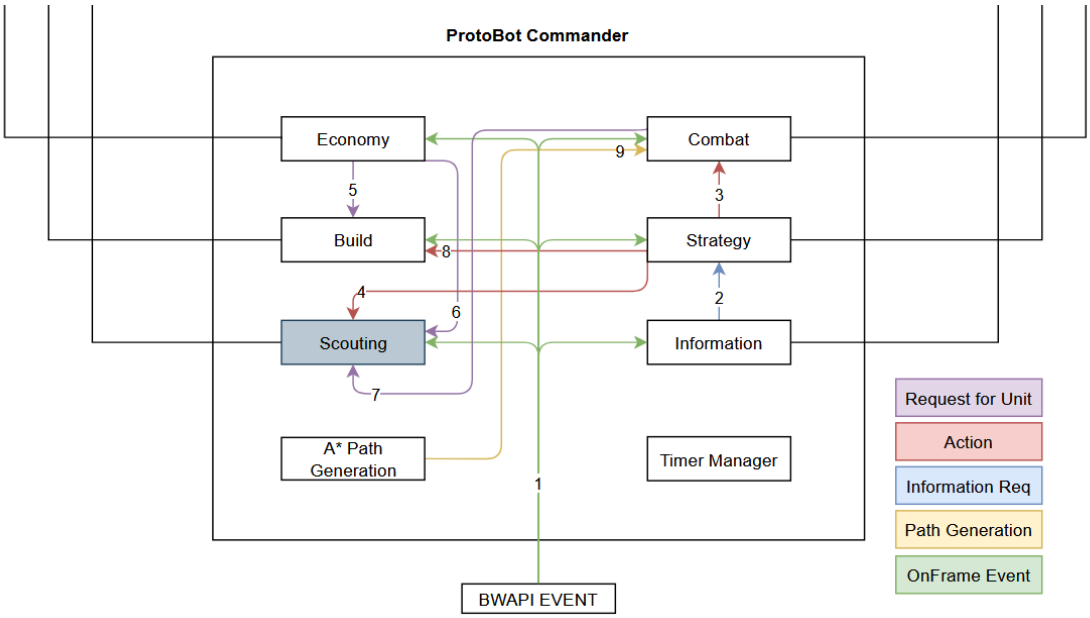

Figure 5: Scouting subsystem diagram

4.3.1 Assumptions

- Information will automatically be updated based on the vision revealed by scouting
- When a unit becomes a scout, it will not be reassigned back to the module it was previously assigned to
- There will be enough resources to perform scouting

4.3.2 Responsibilities

- A scouting action, received from the Strategy Manager through ProtoBot Commander, will prompt the Scouting Manager to choose a suitable unit ProtoBot currently owns to assign as a scout.
- Send BWAPI events to Scouting Manager

4.3.3 Subsystem Interfaces

Table 4: Scouting subsystem interfaces

ID	Description	Inputs	Outputs
#1	StarCraft BWAPI Event	BWAPI Event	Actions to perform scout manipulation
#4	Action from strategy manager to scout	Confirmation of action	Finding suitable unit to scout
#6	Request for a worker (Protoss probe) to scout	Worker	A wrorker scout is created
#7	Request for a combat unit to scout	Combat unit	A Zealot, Dragoon, Observer, or Dark Templar scout is created

4.4 Information

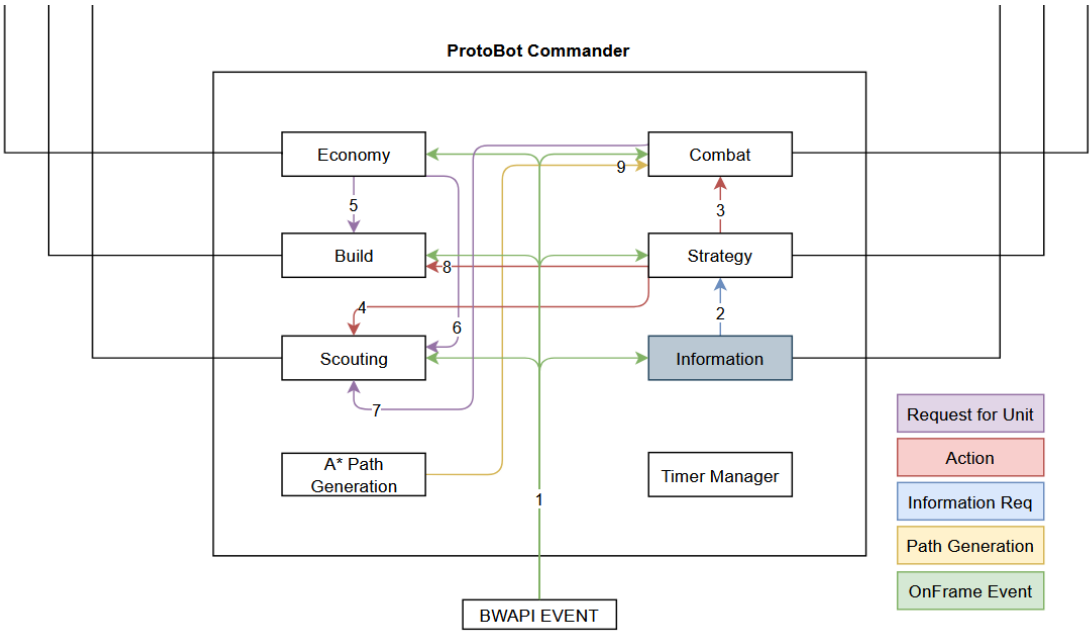

Figure 6: Information subsystem diagram

4.4.1 Assumptions

- Information will be updated over the course of a StarCraft game through BWAPI events
- Outdated information will be removed or marked as outdated
- Information Manager will be globally accessible for all managers

4.4.2 Responsibilities

- Send BWAPI events to Information Manager

4.4.3 Subsystem Interfaces

Table 5: Information subsystem interfaces

ID	Description	Inputs	Outputs
#1	StarCraft BWAPI Event	BWAPI Event	Update information to reflect current game state of StarCraft game and keep track of hidden information

4.5 Strategy

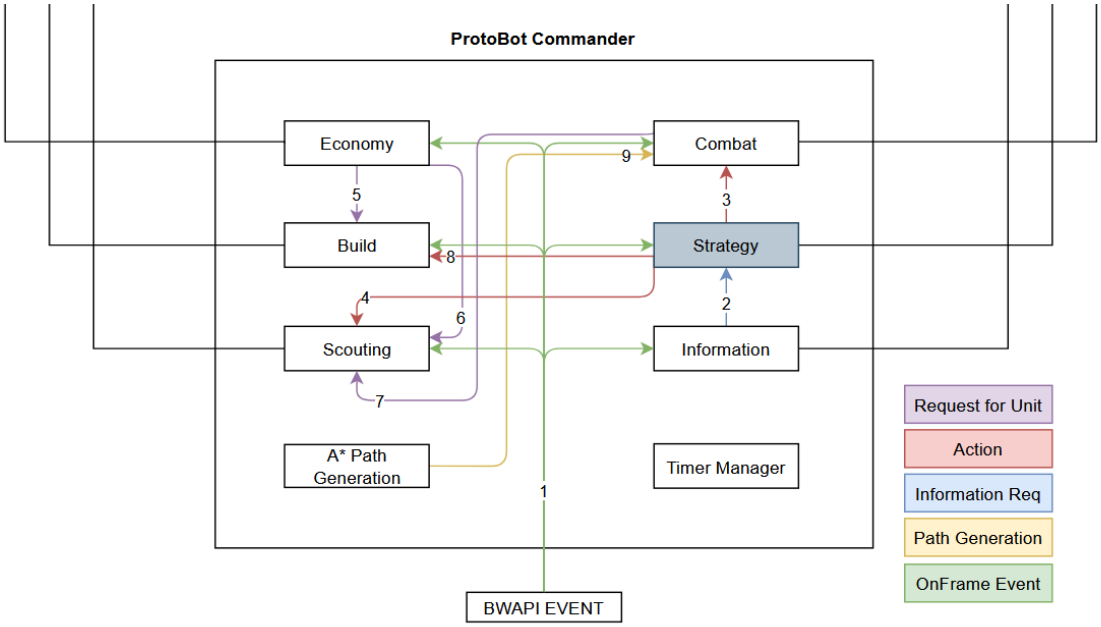

Figure 7: Strategy subsystem diagram

4.5.1 Assumptions

- The strategies used will be the best possible moves ProtoBot can achieve with its current state
- Actions and resource requests will be sent back on the same frame Strategy OnFrame function is called.

4.5.2 Responsibilities

- Send BWAPI events to Strategy Manager
- Service actions returned by the Strategy manager to command other managers

4.5.3 Subsystem Interfaces

Table 6: Strategy subsystem interfaces

ID	Description	Inputs	Outputs
#1	StarCraft BWAPI Event	BWAPI Event	Returns a list of actions for every manager to perform and adds items to resource requests for units and buildings to create
#2	Input for information about the current state of StarCraft game	All information avalible from information manager	Actions that are able to command and build units based on the opponent information

4.6 Combat

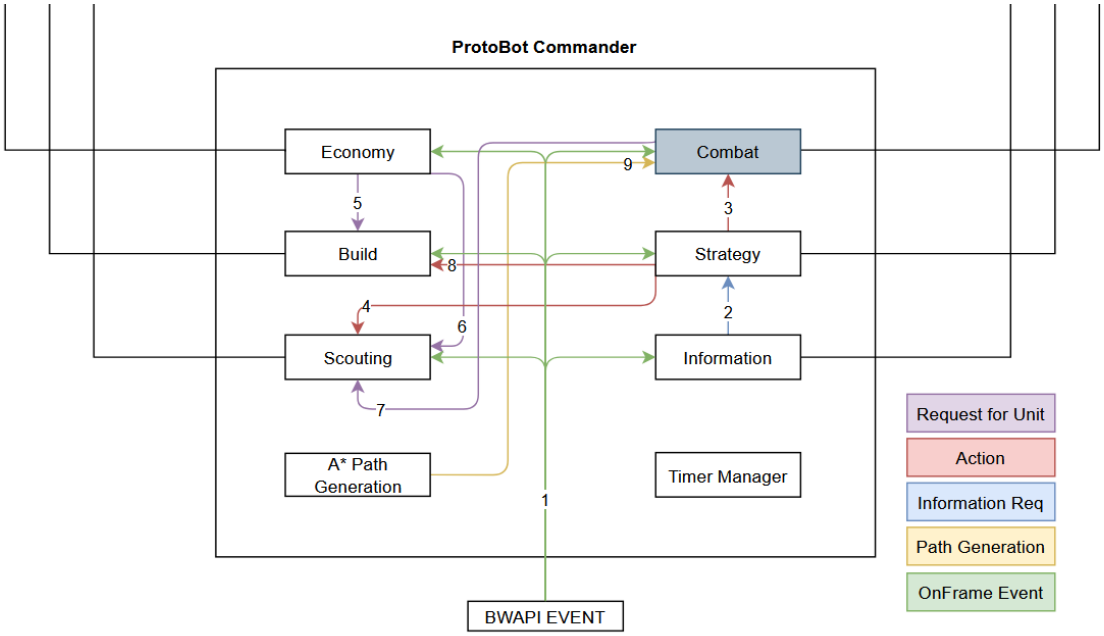

Figure 8: Combat subsystem diagram

4.6.1 Assumptions

- Combat Manager and Strategy Manager will be able to communicate as needed
- Squad unit manipulation will not require any other input besides BWAPI events to occur

4.6.2 Responsibilities

- Send BWAPI events to Combat Manager
- Any combat unit created will be assigned to the Combat Manager

4.6.3 Subsystem Interfaces

Table 7: Combat subsystem interfaces

ID	Description	Inputs	Outputs
#1	StarCraft BWAPI Event	BWAPI Event	Actions that manipulate combat units to attack, defend, and reinforce
#3	Action from Strategy Manager	Attack/Defend/Reinforce action	Squads' state machines updated to attack/move towards enemies or away from them
#9	Generation of Astar path to location	Start position, end position, and unit type	Optimal path to position

4.7 A* Path Generation

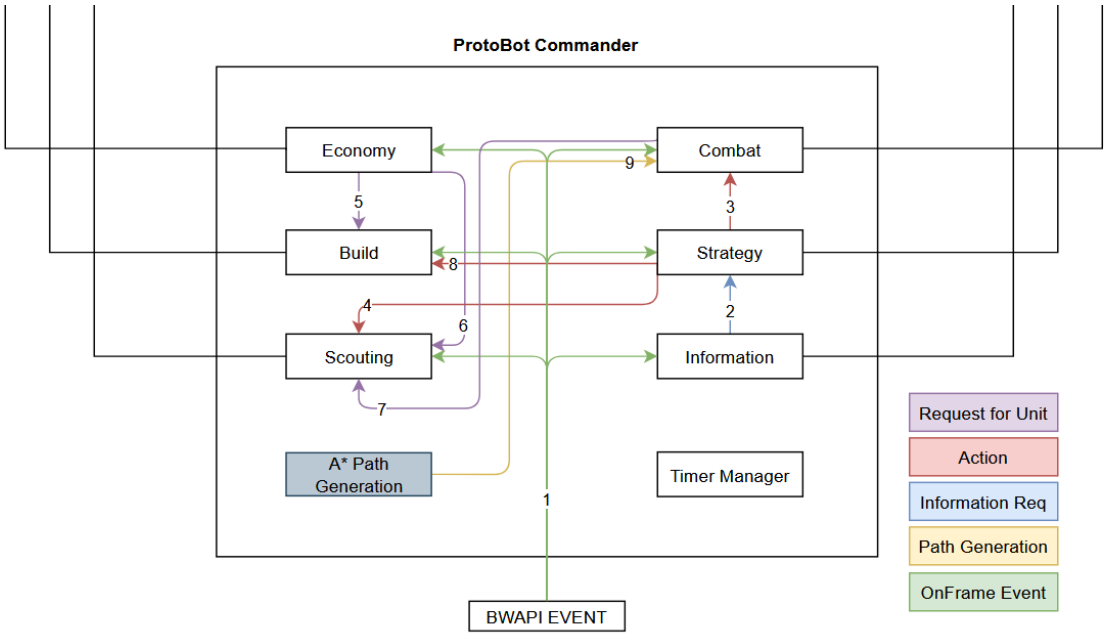

Figure 9: A* Path Generation subsystem diagram

4.7.1 Assumptions

- Path generation will work on all SSCAIT/BASIL competitive maps
- If a path cannot be generated, an empty Path will be returned
- Path generation will not exceed a specified amount of execution time
- Path generation will be statically defined and accessible by any module that needs to use A* path generation

4.8 Timer Manager

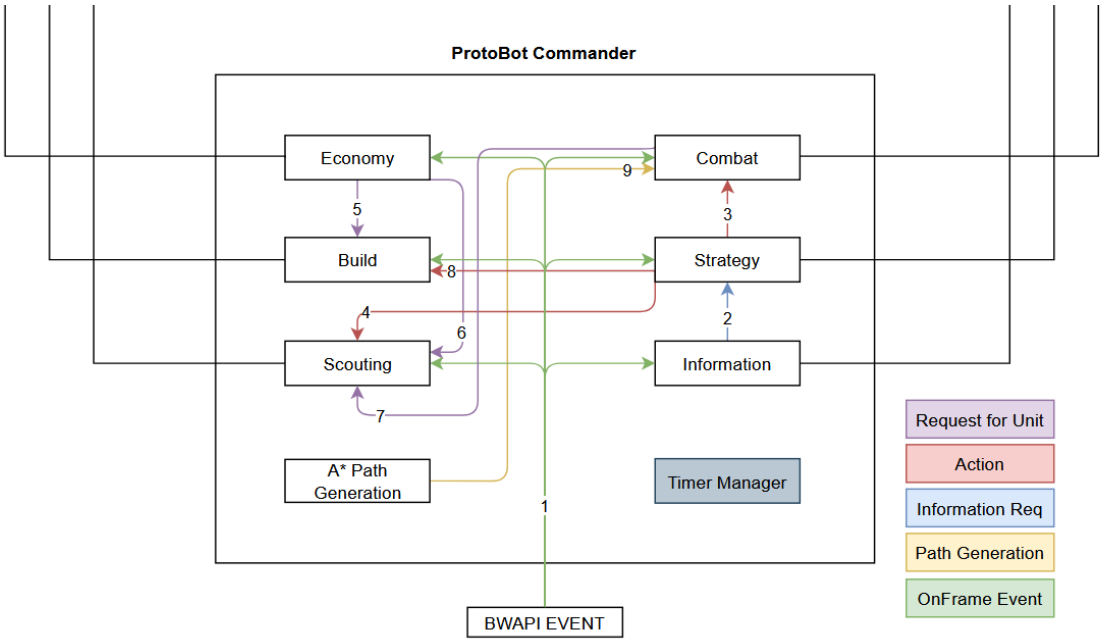

Figure 10: A* Path Generation subsystem diagram

4.8.1 Assumptions

- The timer manager will keep track of the time a module spent executing down to the microsecond
- Track the amount of time taken for execution
- Send a warning if a module takes more than a specified amount of time

4.8.2 Responsibilities

- Start and stop each timer to accurately reflect a manager's OnFrame execution time

5 Economy Manager

The economy manager handles the task of optimally harvesting StarCraft resources. The commander assigns newly created workers to farm resources through the Economy Manager. The Economy Manager controls what base a worker will farm resources at and how many workers are allocated to each base. This manager can also transfer workers to a new base if one is over-saturated. Each base is defined as a nexus economy and is created each time a resource depot (base) is completed.

5.1 Nexus Economy

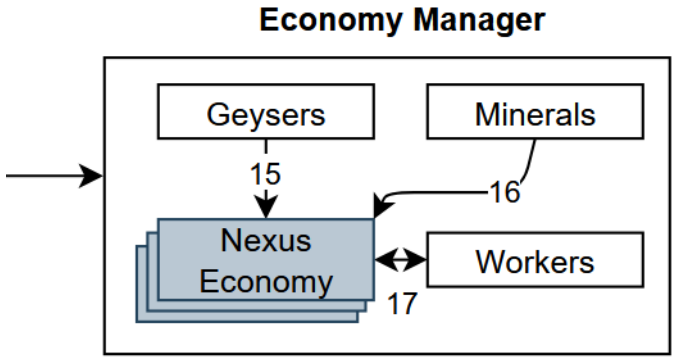

Figure 11: Nexus Economy subsystem diagram

5.1.1 Assumptions

- The number of workers assigned to a base will be the maximum amount defined as

$$OPTIMAL_WORKERS_PER_MINERAL = 3$$

$$WORKERS_PER_ASSIMILATOR = 2$$

$$optimalWorkerAmount = (\#\ of\ minerals\ at\ base) * OPTIMAL_WORKERS_PER_MINERAL$$

$$baseHasGeyser = (Vespene\ geyser\ located\ at\ base\ ?\ WORKERS_PER_ASSIMILATOR : 0)$$

$$maximumWorkers = optimalWorkerAmount + baseHasGeyser$$

- The strategy manager will decide whether training more workers is necessary at a resource depot
- There are no scenarios in which there will be idle workers

5.1.2 Responsibilities

- Manage the assignment of workers to optimally farm resources at a base
- Find workers for scouting and building when needed
- Handle the transfer of workers from one nexus to another in the case of a newly created nexus or a nexus being destroyed
- Determine the resources located at a base when a nexus economy is newly formed

5.1.3 Subsystem Interfaces

Table 8: Nexus Economy subsystem interfaces

ID	Description	Inputs	Outputs
#15	Determine geysers at base	Resource Depot (Nexus)	Search using BWEM to find geysers at base to assign to nexus economy
#16	Determine number of minerals at base	Resource Depot (Nexus)	Assignment of the total minerals located at a base
#17	Assignment of workers	Worker unit (Protoss Probe)	Assignment of worker to a nexus economy

5.2 Geysers

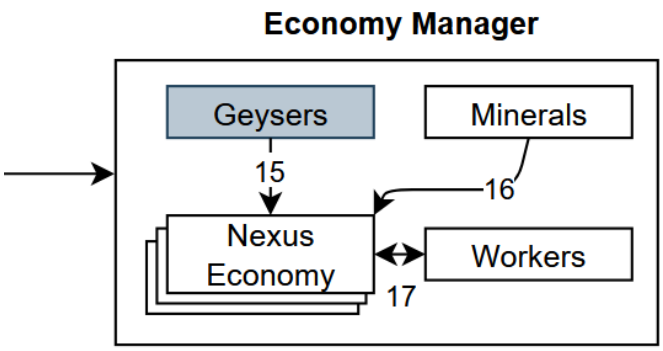

Figure 12: Geysers subsystem diagram

5.2.1 Assumptions

- There will be enough workers available to farm gas
- An assimilator will be created so that workers can drill for gas
- Geyser will be set as null if it does not exist at a base

5.3 Minerals

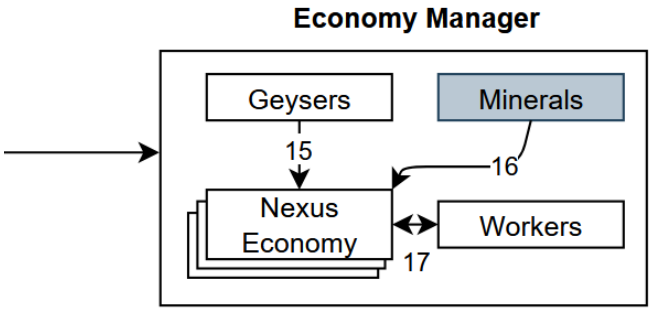

Figure 13: Mineral subsystem diagram

5.3.1 Assumptions

- There will be enough workers available to farm minerals

5.3.2 Responsibilities

- Send idle workers to the closest minerals for harvesting
- Increase the number of workers per mineral as the number of workers increases
- Transfer workers assigned to mineral when depleted

5.4 Workers

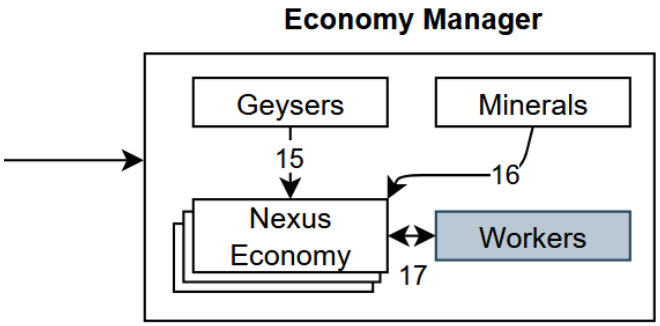

Figure 14: Worker subsystem diagram

5.4.1 Assumptions

- The Commander will provide the Economy manager with units to assign to each Nexus Economy
- New workers will constantly be trained and given to the Economy manager
- When a worker is assigned to farm a resource, that resource will be the most optimal resource for that worker to farm

5.4.2 Responsibilities

- Execute actions specified by Nexus Economy to perform optimal resource gathering.

5.4.3 Subsystem Interfaces

Table 9: Workers subsystem interfaces

ID	Description	Inputs	Outputs
#17	Action for worker to perform to gather/drill a resource	Mineral/Refinery to select, specific worker to command	Worker performs specified action

6 Build Manager

The Build Manager is responsible for managing building placement and build order at the beginning of a game. The Build Manager will select a build order based on the specific race of the opponent, coordinate the construction of buildings through builder units, and determine the placement of where a building should be construed through the Building Placer.

6.1 Build Order

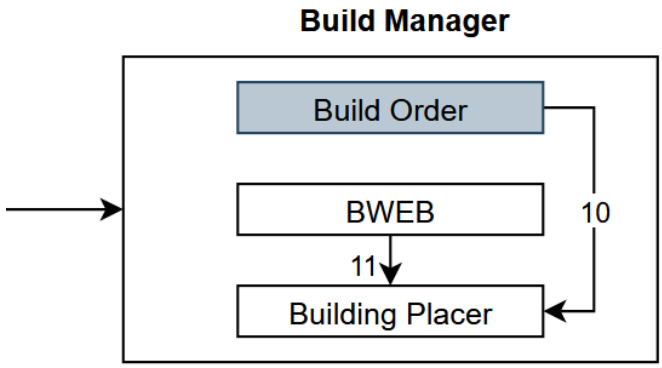

Figure 15: Build Order subsystem diagram

6.1.1 Assumptions

- The Economy Manager is working properly to provide necessary resources for units to be created
- The Strategy Manager does not conflict with the buildings in a build order.

6.1.2 Responsibilities

- Set up a build order depending on the enemy's chosen race
- Decide specific scout timings depending on build order

6.2 BWEB Module

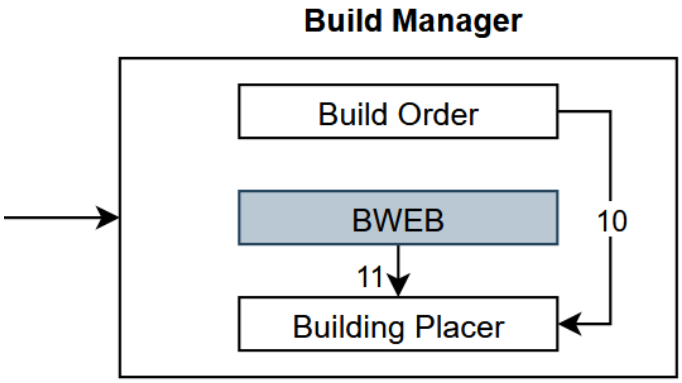

Figure 16: BWEB Module subsystem diagram

6.2.1 Assumptions

- The module is fully initialized and functions correctly upon game start
- There is sufficient space on the map for valid building placements

6.2.2 Responsibilities

- Generate building placements using the BWEM
- Generate choke-point locations around the map

6.3 Building Placer

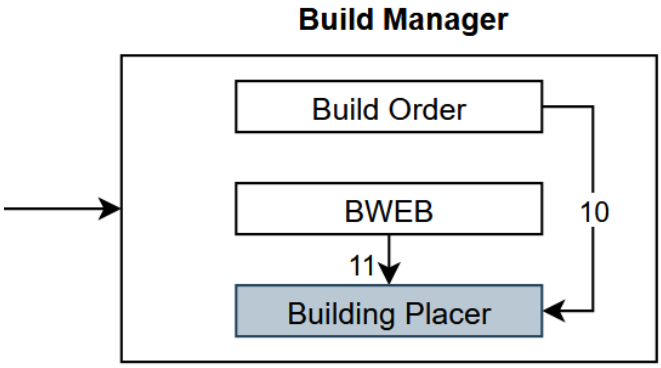

Figure 17: Building Placer subsystem diagram

6.3.1 Assumptions

- The module is receiving valid building requests from the Strategy Manager
- The BWEB module and other map-analysis tools are properly initialized
- Placement checks can be performed without significantly affecting game performance

6.3.2 Responsibilities

- Determine valid tile positions for requested buildings
- Reserve placements of buildings for requests already approved at a tile position
- Select building placements for efficient base layout and proper pylon power coverage
- Place buildings as close to ProtoBot's main base and as far away from the opponent's main base as possible.

6.3.3 Subsystem Interfaces

Table 10: Building Placer subsystem interfaces

ID	Description	Inputs	Outputs
#11	Interface with BWEB data	BWEM initialization	Generated building placements and choke points
#10	Buildings from Build Order	Planned building	Placement for building

7 Scout Manager

The Scout Manager is responsible for having units travel around the map to reveal information hidden due to the fog of war. The Scout Manager uncovers areas covered by fog and will perform specific scouting behaviors based on the type of unit a scout is defined as. When information is discovered by the Scout Manager, the Information manager will automatically update any new information learned.

7.1 Scout Module

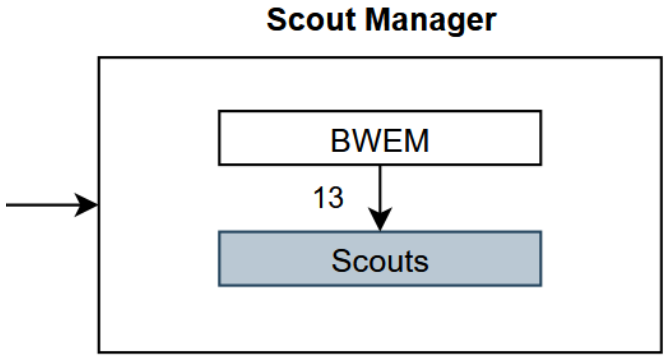

Figure 18: Scout Module subsystem diagram

7.1.1 Assumptions

- Information will be automatically updated based on the information a scout reveals
- There will be enough resources to perform scouting

7.1.2 Responsibilities

- Perform different behaviors based on the type of unit a scout is (a scout can be a Probe, Zealot, Dragoon, Observer, or Dark Templar)

7.1.3 Subsystem Interfaces

Table 11: Scouts subsystem interfaces

ID	Description	Inputs	Outputs
#13	BWEM to be able to travel around the map	BWEM starting locations and cached BWEM paths	Scouting actions that allow scouts to travel around the map

7.2 BWEM Module

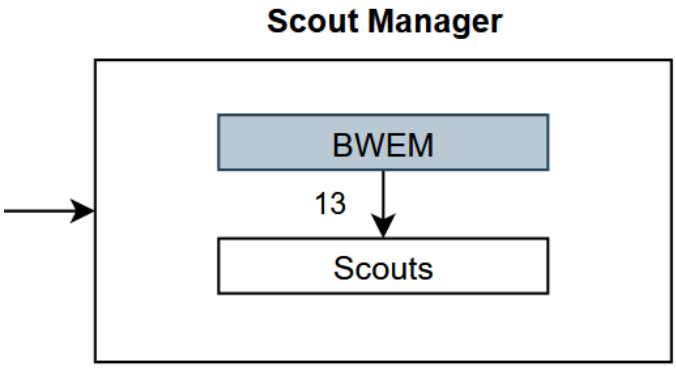

Figure 19: BWEM Module subsystem diagram

7.2.1 Assumptions

- BWEM is compatible with ProtoBot
- BWEM is properly initialized at the beginning of the game

7.2.2 Responsibilities

- Analyze the selected map for the current StarCraft match
- Convert analysis into readable data

8 Information Manager

The Information Manager adds and maintains information about the state of the game by collecting data from scouting, combat, and other subsystems that command units around the map. This manager also maintains a Unit/Building Tracker, which stores detailed and persistent records of friendly and enemy units and structures, including their unit types and last known positions.

8.1 Unit/Building Tracker

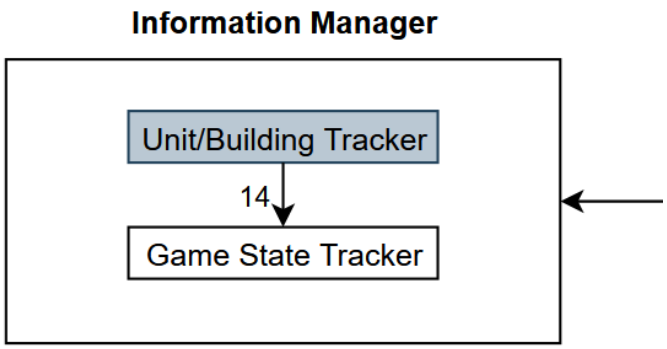

Figure 20: Unit/Building Tracker subsystem diagram

8.1.1 Assumptions

- Information on visible units and buildings will be updated during gameplay
- Previously seen enemy units and buildings can be stored even when they are no longer visible
- Tracking units and buildings will not significantly reduce ProtoBot's execution time

8.1.2 Responsibilities

- Update all information about the opponent.
- Maintain a record of previously seen enemy units and structures
- Remove or mark outdated information

8.1.3 Subsystem Interfaces

Table 12: Unit/Building Tracker subsystem interfaces

ID	Description	Inputs	Outputs
#14	Update data structures used for tracking units and buildings	Enemy appears in ProtoBot's view	Unit or building added to appropriate data structure

8.2 Game State Tracker

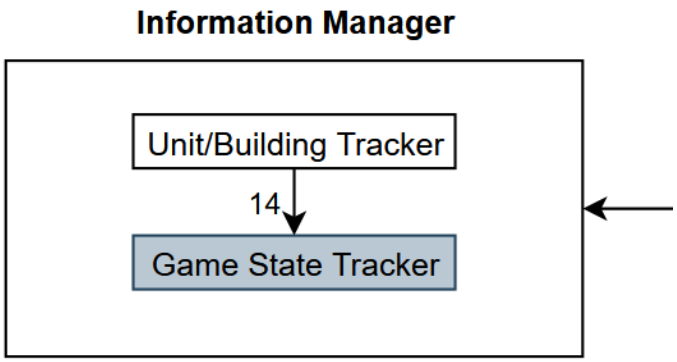

Figure 21: Game State Tracker subsystem diagram

8.2.1 Assumptions

- Game state information will be updated prior to it being used
- Constant monitoring of the game will not slow down execution time

8.2.2 Responsibilities

- Recognize enemy's current strategy and build order upon receiving information, such as early rush or fast expand

8.2.3 Subsystem Interfaces

Table 13: Game State Tracker subsystem interfaces

ID	Description	Inputs	Outputs
#14	Determine the strategy an opponent is trying to attempt	Units created by opponent	Determined strategy being attempted

9 Strategy Manager

The Strategy Manager selects the best actions for the ProtoBot to take. This manager utilizes the Information Manager to make decisions and commands squads to attack, defend, and reinforce locations around the map and create units based on the resources available.w

9.1 Strategy State

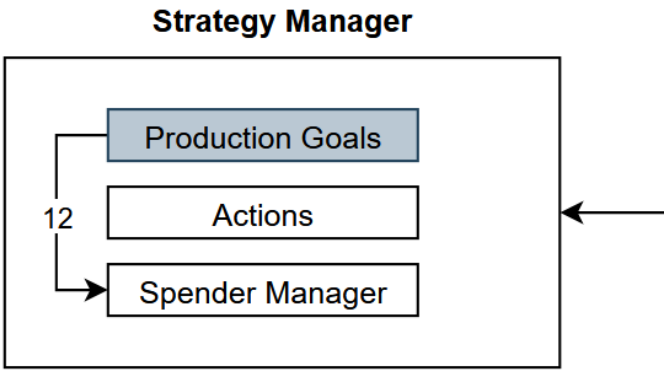

Figure 22: Strategy State subsystem diagram

9.1.1 Assumptions

- The information manager will be updated before the strategy managers runs

9.1.2 Responsibilities

- Change production goals based on the opponent race and units already created
- Signal to create a specified amount of units
- Determine when ProtoBot should research upgrades

9.2 Production Goal

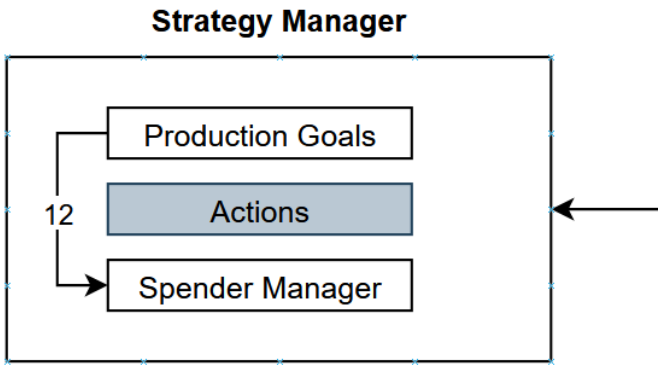

Figure 23: Production Goal subsystem diagram

9.2.1 Assumptions

- ProtoBot has updated all the information necessary to create valid actions

9.2.2 Responsibilities

- Specify unit training and upgrade priorities/timing
- Determine building priorities using heuristic rules
- Command squads to defend choke points or attack opponent units

9.3 Spender Manager

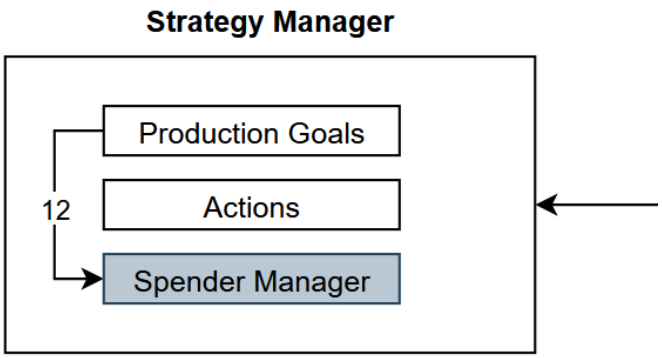

Figure 24: Spender Manager subsystem diagram

9.3.1 Assumptions

- Resources are being optimally farmed by the Economy Manager
- The strategy manager provides valid resource requests to spend

9.3.2 Responsibilities

- Decide how to allocate resources based on production goals
- Reserve resources for buildings based on resource requests given

9.3.3 Subsystem Interfaces

Table 14: Spender Manager subsystem interfaces

ID	Description	Inputs	Outputs
#12	Verify purchase of units for production goals	Flattened production goals ProtoBot can achieve	reserving of resources for a request

10 Combat Manager

The combat manager is responsible for the manipulation of ProtoBot's combat units. When a combat unit is created, the unit will automatically be assigned the combat manager. In order for the manipulation of units to be straightforward, combat units are separated in groups called squads. Each squad then is able to perform combat behaviors based on a sate machine that is updated based on the actions received from strategy.

10.1 Squads

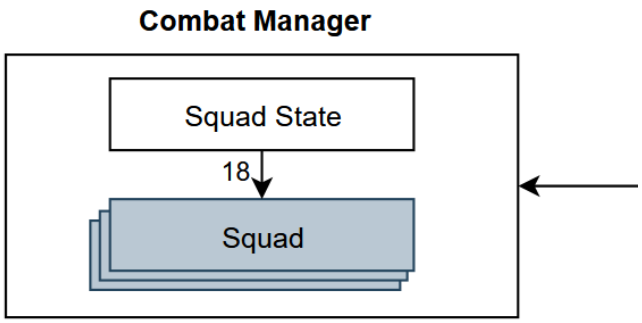

Figure 25: Combat Module subsystem diagram

10.1.1 Assumptions

- There will be enough units available for squads
- Actions received from Strategy Manager will be valid positions for a squad to move to or attack
- A Squad will always be a group of 12 combat units
- When a combat unit dies, its squad will correctly handle the memory of the unit to update the squad's information
- When a squad dies, the Combat Manager will correctly handle the squad's deletion in memory

10.1.2 Responsibilities

- Handle the manipulation of units to kite
- Move units around the map based on the action of a leader
- Manipulate squads to fight close together
- Handle the manipulation of squads through terrain

10.1.3 Subsystem Interfaces

Table 15: Squad Module subsystem interfaces

ID	Description	Inputs	Outputs
#18	Squad State Machine	Actions from Strategy Manager or squad being attacked	State change caused by input or squad behavior based on the current state

10.2 Squad State

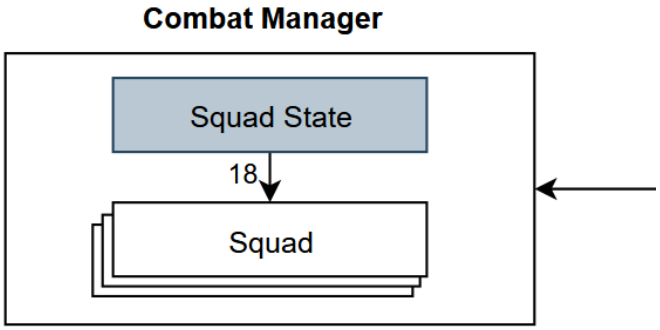

Figure 26: Squad Module subsystem diagram

10.2.1 Assumptions

- A squad's state machine will enter and exit different states when necessary
- The current state of a squad accurately reflects the action requested by Strategy Manager

10.2.2 Responsibilities

- Manipulate units in squad based on current state
- Correctly transition between states at appropriate times

References